

MovieAgentOS

Sistema Multi-Agent per Raccomandazioni Personalizzate di Film

Relazione Tecnica

Progetto per il corso di Applicazioni Intelligenti — A.A. 2025/2026

Autori:

- YASSINE EL HOSAYNY — Matricola: 20053157
- KEVIN KOLAVERI — Matricola: 20049523
- ALBERTO RODOLFO CULLA — Matricola: 20044538

1. Scopo del Documento

Il presente documento descrive la visione, gli obiettivi e i requisiti principali del programma intitolato: MovieAgentOS, un sistema di raccomandazione cinematografica basato su un'architettura multi-agent. L'utente dialoga in linguaggio naturale con un'interfaccia chat e riceve suggerimenti di film personalizzati in base alle proprie preferenze, alla cronologia della sessione e ai feedback forniti in precedenza.

2. Obiettivi del Prodotto

2.1 Obiettivi principali

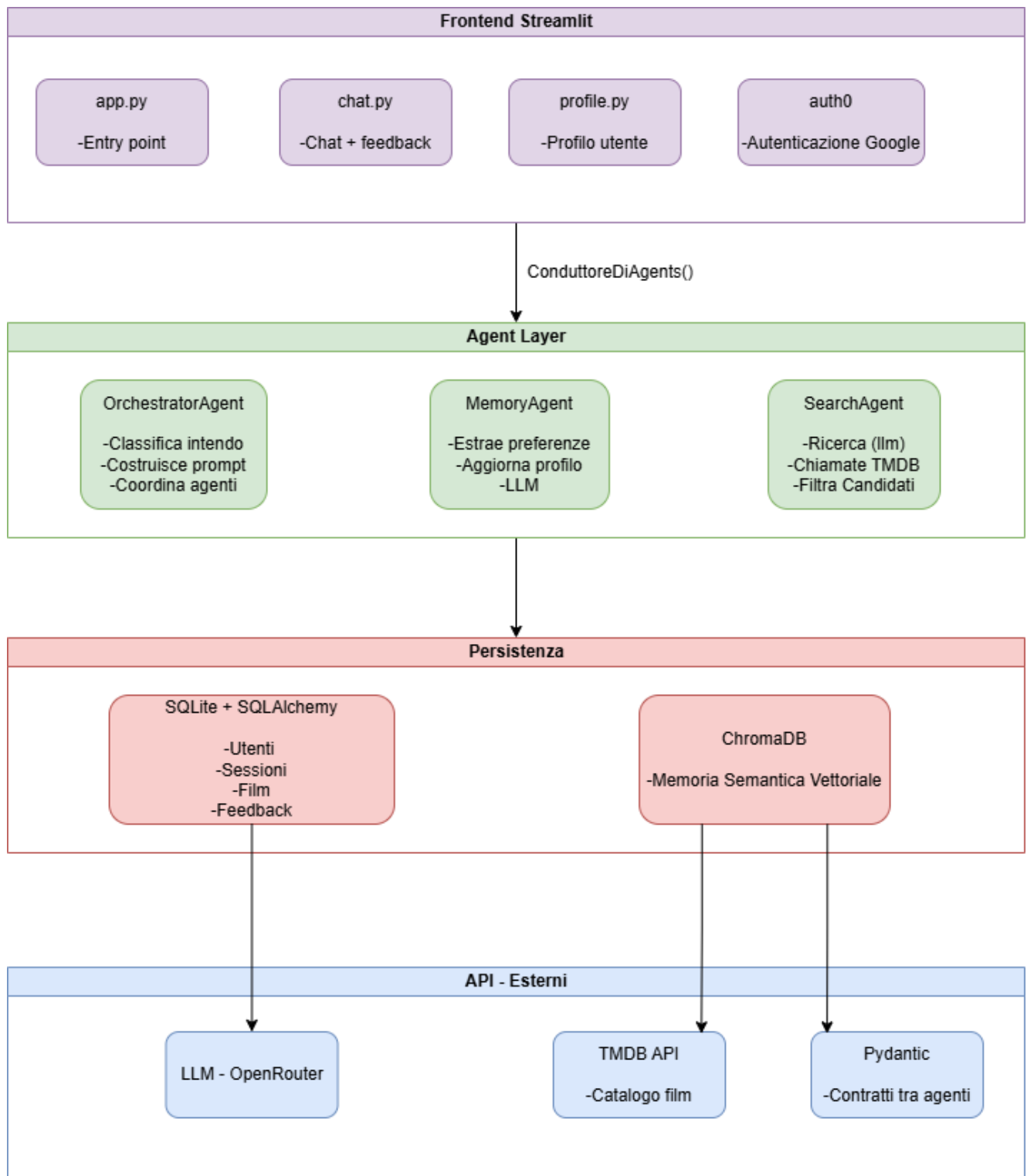
- Fornire raccomandazioni di film reali (catalogo TMDB), coerenti e personalizzate.
- Memorizzare le preferenze dell'utente sia in forma strutturata (SQLite) sia semantica (ChromaDB) in modo persistente tra le sessioni.
- Rispettare i vincoli espliciti espressi in linguaggio naturale: genere, voto minimo, anno, esclusioni, similitudini.
- Non proporre film già visti o già consigliati nella stessa sessione.

3. Architettura Generale

Il sistema è organizzato in tre strati principali:

- Interfaccia utente con Streamlit .
- Orchestrazione degli agenti con linguaggio Python.
- Persistenza dei dati, il flusso di una richiesta attraversa sempre lo stesso percorso, ma il sottoinsieme di agenti attivati varia in base alla necessità.

Tecnologie principali	Python, Streamlit, agno Agent SDK
LLM	OpenRouter — openai/gpt-oss-20b:free
Database relazionale	SQLite + SQLAlchemy ORM
Memoria semantica	ChromaDB
Catalogo film	TMDB API
Autenticazione	Auth0 / Google OAuth



3.1 Stakeholder

Stakeholder	Ruolo	Interessi
Persone	Utenti finali	Conoscere nuovi film in base ai propri gusti e interessi
Team di sviluppo	Sviluppatore	Realizzare un prodotto completo e funzionante
Docente	Supervisore	Valutazione e controllo qualità

3.2 Componenti principali

Componente	Modulo	Responsabilità
OrchestratorAgent	orchestratorAgents.py	Router LLM come decisore primario: classifica l'intento, dialoga con l'utente, compila la RichiestaRicerca e coordina gli altri agenti
MemoryAgent	agente_memoria.py	Estrae e aggiorna preferenze di genere e fatti personali
SearchAgent	agente_ricerca.py	Cerca film reali su TMDb con strategie diverse (genere, tema, simili, titolo, popolari)
Reranker	agente_ricerca.py	Sceglie i migliori candidati reali (scoring deterministico + rerank LLM), con motivazioni
Database layer	backend/db/	SQLite per dati strutturati, ChromaDB per memoria vettoriale
Frontend	frontEnd/	Streamlit chat + pagina profilo
Linguaggio	linguaggio.py	Estrattori deterministici da linguaggio naturale italiano: generi, voto, anno, film di riferimento

4. Installazione

Una volta clonato il repository o scaricato il file zip si avrà la cartella: MovieAgentOS

```
cd MovieAgentOS
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
cd MovieAgentOS
cd MovieAgentOS-main
streamlit run frontend/app.py
```

4.1 Struttura delle directory

Percorso	Contenuto
frontEnd/app.py	Entry point Streamlit
frontEnd/pages/	Pagine: chat.py, profile.py
frontEnd/gui/	Componenti UI riusabili (auth, sidebar, landing)
backend/agents/	Agenti: orchestratore, memoria, ricerca, schemi, llm_config
backend/db/	models.py, database.py, crud.py, file SQLite
backend/db/chroma/	Dati persistenti ChromaDB
backend/tools/	tmdb_tools.py, chroma_tools.py

4.2 Secrets e configurazione

La configurazione sensibile (chiavi API, credenziali Auth0) è gestita come segue:

- `API_KEY_LLM` (o `OPENROUTER_API_KEY`): chiave per l'accesso ai modelli LLM via OpenRouter.
- `MODEL_ID_LLM`: (opzionale) override del modello, default `openai/gpt-oss-20b:free`.
- `TMDB_API_KEY`: chiave per le chiamate al catalogo TMDB.
- Credenziali Auth0: `client_id`, `client_secret`, `domain`, `redirect_uri` per il flusso OAuth2.

4.3 Dipendenze

Dipendenza	Ruolo
streamlit	Framework interfaccia web e gestione stato
sqlalchemy	ORM per SQLite
requests	Chiamate HTTP a TMDB API
chromadb	Database vettoriale per memoria semantica
pydantic	Validazione e tipizzazione dei dati tra agenti
agno	Agent SDK per creazione agenti LLM strutturati
openai / google-genai	Client LLM usati dagli agenti agno (OpenRouter ed eventuali modelli Google)
authlib / httpx	Flusso OAuth2 per l'autenticazione (Auth0, Google)

4.4 Flusso di una richiesta

Ogni messaggio dell'utente percorre il seguente flusso:

- Il frontend (`chat.py`) costruisce lo storico dei messaggi e lo stato di sessione (la `RichiestaRicerca` precedente e i contatori di chiarimenti e raffinamenti) e invoca `ConduttoreDiAgents`.

- Il router LLM è il decisore primario: classifica l'azione (raccomandazione, chiarimento, conversazione, aggiornamento memoria, fuori dominio) e il tipo di richiesta (NUOVA / CONTINUAZIONE / RAFFINAZIONE) e compila una RichiestaRicerca strutturata.
- Il codice Python valida la decisione (generi TMDB reali, vincoli numerici, estrattori deterministici), la fonde con lo stato di sessione e applica il profilo e i feedback dell'utente.
- Se la richiesta implica un aggiornamento di preferenze o un feedback, viene chiamato il MemoryAgent, che aggiorna SQLite e ChromaDB.
- Se la richiesta implica una ricerca, il SearchAgent riceve la RichiestaRicerca strutturata, raccoglie un pool di film reali da TMDB, applica i vincoli in un solo punto e sceglie i migliori con scoring deterministico e rerank LLM.
- Se i candidati validi sono ancora troppi, l'orchestratore pone UNA domanda di raffinamento e salva il pool nello stato di sessione; altrimenti compone la risposta con titolo, anno, voto e motivazione per ogni film.
- Il frontend salva il messaggio assistente e i film consigliati su SQLite, aggiorna lo stato di sessione e li mostra con widget di feedback.

5. Frontend

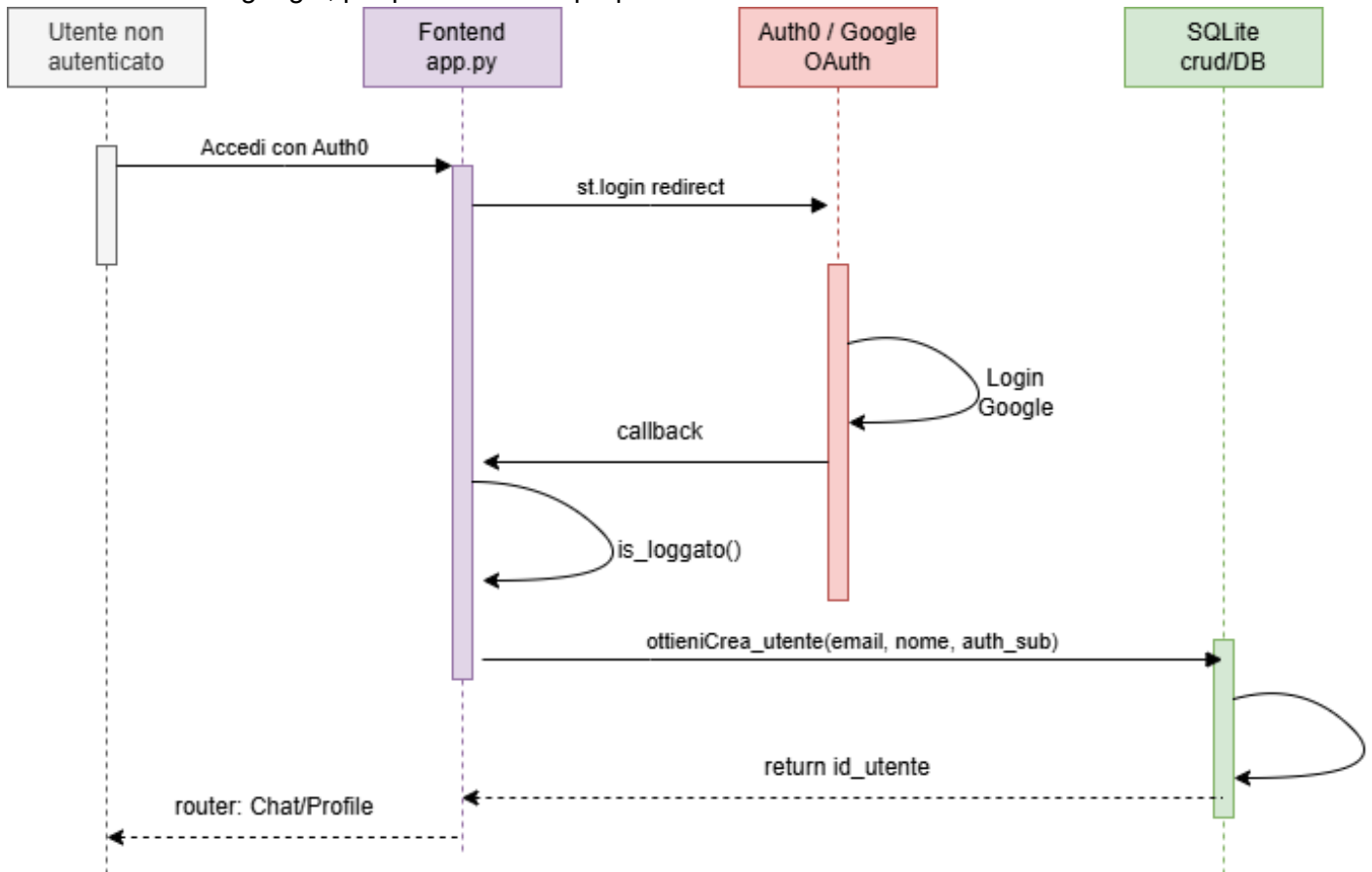
Il frontend è realizzato interamente con Streamlit.

5.1 Struttura del frontend

File	Percorso	Funzione
app.py	frontEnd/app.py	Entry point: inizializza il DB, controlla autenticazione, instrada alle pagine
chat.py	frontEnd/pages/chat.py	Pagina principale: chat con l'agente, gestione sessioni, feedback film
profile.py	frontEnd/pages/profile.py	Pagina profilo utente: visualizza e cancella preferenze
auth.py	frontEnd/gui/auth.py	Helper is_loggato(): verifica stato login tramite st.user
mostra_pagina_loggato.py	frontEnd/gui/	Sidebar con avatar, nome utente e pulsante logout
mostra_pagina_iniziale.py	frontEnd/gui/	Landing page per utenti non autenticati con pulsante Auth0

5.2 Autenticazione

L'autenticazione utilizza il provider Auth0 per la creazione di account ed è integrata con l'autenticazione google, per poter usare la propria mail.



6. OrchestratorAgent (ConduttoreDiAgents)

Il modulo orchestratorAgents.py contiene il cuore della logica di coordinamento. La funzione principale ConduttoreDiAgents è l'unico punto di ingresso chiamato dal frontend.

6.1 Classificazione dell'intento (router LLM)

La funzione `decidi_router` invia al modello LLM (tramite `agno`) il messaggio corrente, lo storico e la memoria, e ottiene una `DecisioneRouter`: azione, tipo di richiesta (NUOVA / CONTINUAZIONE / RAFFINAZIONE) e una bozza di `RichiestaRicerca` strutturata.

Il router è il decisore primario, ma non ha l'ultima parola: il codice Python valida e corregge la sua decisione, così da non delegare interamente il comportamento del sistema al modello.

6.2 Validazione e fusione con lo stato di sessione

Dopo il router, alcune funzioni deterministiche completano e correggono la decisione:

- `valida_decisione`: valida azione e tipo, filtra i generi tenendo solo quelli reali di TMDb e recupera dal testo i vincoli espliciti (voto, anno, esclusioni) tramite gli estrattori di `linguaggio.py`.

- `correggi_decisione`: correzioni generali (una raffinazione che aggiunge solo vincoli non riparte da zero, un cambio di genere è una richiesta NUOVA, un solo fatto personale aggiorna la memoria e non cerca).
- `fondi_con_precedente`: una CONTINUAZIONE riparte dalla richiesta precedente (stessa ricerca, pagina successiva), una RAFFINAZIONE vi aggiunge i nuovi vincoli senza perdere quelli già raccolti.

Infine `applica_profilo` e `applica_gusti_feedback` aggiungono le preferenze salvate: i generi da evitare come vincolo (sospeso per un genere chiesto esplicitamente ora), i generi preferiti e i feedback come preferenze morbide per il rerank.

6.3 Gestione dei film da evitare

Prima di invocare il SearchAgent, l'orchestratore costruisce l'insieme dei film da escludere, usando come vincolo forte i `tmdb_id` (dato preciso da SQLite):

- I film che l'utente dice di aver già visto o di non volere; se dice "li ho già visti", gli ultimi consigli vengono salvati come visti (`segna_ultimi_consigli_come_visti`).
- I film già consigliati nella sessione corrente, recuperati da SQLite.
- I film marcati come già visti nei feedback e nella tabella FilmVisto dell'utente (fonte precisa e persistente tra le sessioni).

Questi `tmdb_id` (e i titoli di riferimento) vengono passati al SearchAgent come vincolo hard: i film corrispondenti non compaiono mai nell'output finale. Se l'utente indica quale film ha scelto, `segna_film_scelto` lo salva in SQLite come visto e gradito.

7. MemoryAgent

Il MemoryAgent (`agente_memoria.py`) gestisce l'estrazione e il salvataggio delle preferenze dell'utente.

7.1 Estrazione delle preferenze

Il modulo combina due approcci complementari:

- Estrazione rule-based (`estrai_memoria_semplice`): regex e pattern linguistici italiani per rilevare il nome dell'utente, liste di generi preferiti e generi da evitare. Questo approccio è veloce e non consuma chiamate LLM per i casi più semplici.
- Estrazione LLM (MemoryAgent via `agno`): per messaggi ambigui o complessi che non vengono catturati dalle regex, viene invocato il modello con istruzioni strutturate per estrarre `generi_preferiti`, `generi_da_evitare`, `summary_memoria` e `confidence`.

I due risultati vengono unificati: le preferenze positive e negative estratte dalle regex vengono fuse con quelle dell'LLM, con deduplicazione per chiave normalizzata, così "mi piacciono commedia e thriller ma non horror" salva correttamente sia i generi preferiti sia quelli da evitare.

7.2 Persistenza in SQLite e ChromaDB

Una volta estratte le preferenze, il MemoryAgent le salva su due livelli:

- SQLite (`ProfiloUtente`): `generi_preferiti` e `generi_da_evitare` come array JSON, più `summary_testuale` e `preferenze_json`. È la fonte precisa e autorevole per le preferenze di genere. I feedback (thumbs) e i film visti sono salvati nelle rispettive tabelle (`Feedback`, `FilmVisto`).

- ChromaDB (memoria semantica): il testo del messaggio viene salvato come documento vettoriale con metadati (tipo, fonte, timestamp). Questo permette in futuro di recuperare memorie per similarità semantica.

8. SearchAgent e Reranker

Il SearchAgent (agente_ricerca.py) è responsabile del recupero di film reali da TMDB. Riceve dal conduttore una RichiestaRicerca strutturata (nessun testo da ri-analizzare), raccoglie un ampio pool di candidati che rispettano i vincoli duri e sceglie i migliori con un rerank in due passi.

L'LLM interviene solo nel rerank per scegliere tra i candidati reali; la raccolta e i vincoli sono deterministici. Sulle richieste specifiche (genere, tema, simili, titolo) non si usano mai fallback popolari: meglio zero risultati dichiarati che tre film sbagliati.

8.1 Raccolta dei candidati

In base al campo tipo della RichiestaRicerca, raccogli_candidati sceglie la strategia di raccolta:

- genere: /discover/movie filtrato per gli ID TMDB dei generi richiesti.
- tema: la query tematica viene tradotta in keyword TMDB e cercata con /discover/movie.
- simili: si individua il film di riferimento e si usano le sue raccomandazioni TMDB.
- titolo: ricerca per titolo di un film specifico su /search/movie.
- generica: film popolari con i soli vincoli espliciti (fallback dichiarato nella risposta).

I vincoli numerici (voto, anno) e le esclusioni non sono delegati all'LLM: sono estratti dal testo dal codice (linguaggio.py) e applicati in un unico punto, film_rispetta_vincoli.

La scelta della strategia spetta al router del conduttore, non al SearchAgent: quest'ultimo esegue la RichiestaRicerca e non re-interpreta mai testo libero, evitando ricerche fuori contesto.

8.2 Strategie di ricerca

Strategia	Endpoint TMDB	Logica
genere	/discover/movie	Filtra per ID genere, voto minimo, anno, ordine per popolarità
simili	/movie/{id}/recommendations	Recupera film simili al film di riferimento, poi filtra per generi del film base
tema	/search/keyword + /discover/movie	Traduce il tema in keyword TMDB, poi filtra per voto e anno
popolari	/movie/popular	Fallback per richieste molto generiche, segnalato nella risposta
titolo	/search/movie	Ricerca o scheda di un film specifico indicato per titolo

8.3 Ricerca "simili a"

La gestione delle richieste "simili a X" è particolarmente curata. Il flusso è:

- Il titolo del film di riferimento viene compilato dal router nel campo `film_base`, oppure estratto dal testo da `estrai_film_riferimento` (`linguaggio.py`).
- Viene effettuata una ricerca TMDb per trovare l'ID del film base.
- Viene chiamato `/movie/{id}/recommendations` (non `/similar`, che tende a restituire risultati troppo generici).
- I candidati vengono filtrati per genere: si conservano solo i film che condividono almeno un genere con il film di riferimento.
- Il film di riferimento stesso viene escluso dai risultati.

Se dopo il filtro per genere non rimangono abbastanza candidati, il sistema preferisce restituire zero risultati piuttosto che proporre film non pertinenti.

8.4 Normalizzazione e scoring

Tutti i risultati TMDb vengono normalizzati in oggetti `FilmCandidato` (schema Pydantic) con campi: `tmdb_id`, `titolo`, `anno`, `descrizione`, `generi`, `voto_medio`, `poster_path`, `numero_voti`, `motivo_ricerca`. Il `motivo_ricerca` registra perché il film è stato scelto, utile per la trasparenza nella risposta finale.

Il rerank avviene in due passi: `punteggio_film` assegna un punteggio deterministico (vincoli soddisfatti e preferenze morbide del profilo), poi `rerank_llm` sceglie tra i soli candidati reali (validazione per `tmdb_id`) i più adatti all'utente, producendo le motivazioni e un commento conversazionale. I film nella lista da evitare (per `tmdb_id`) sono esclusi a monte; senza LLM si ricade sui migliori candidati deterministici.

9. Database

Il sistema usa due meccanismi di persistenza complementari: SQLite per dati strutturati precisi e ChromaDB per memoria semantica vettoriale.

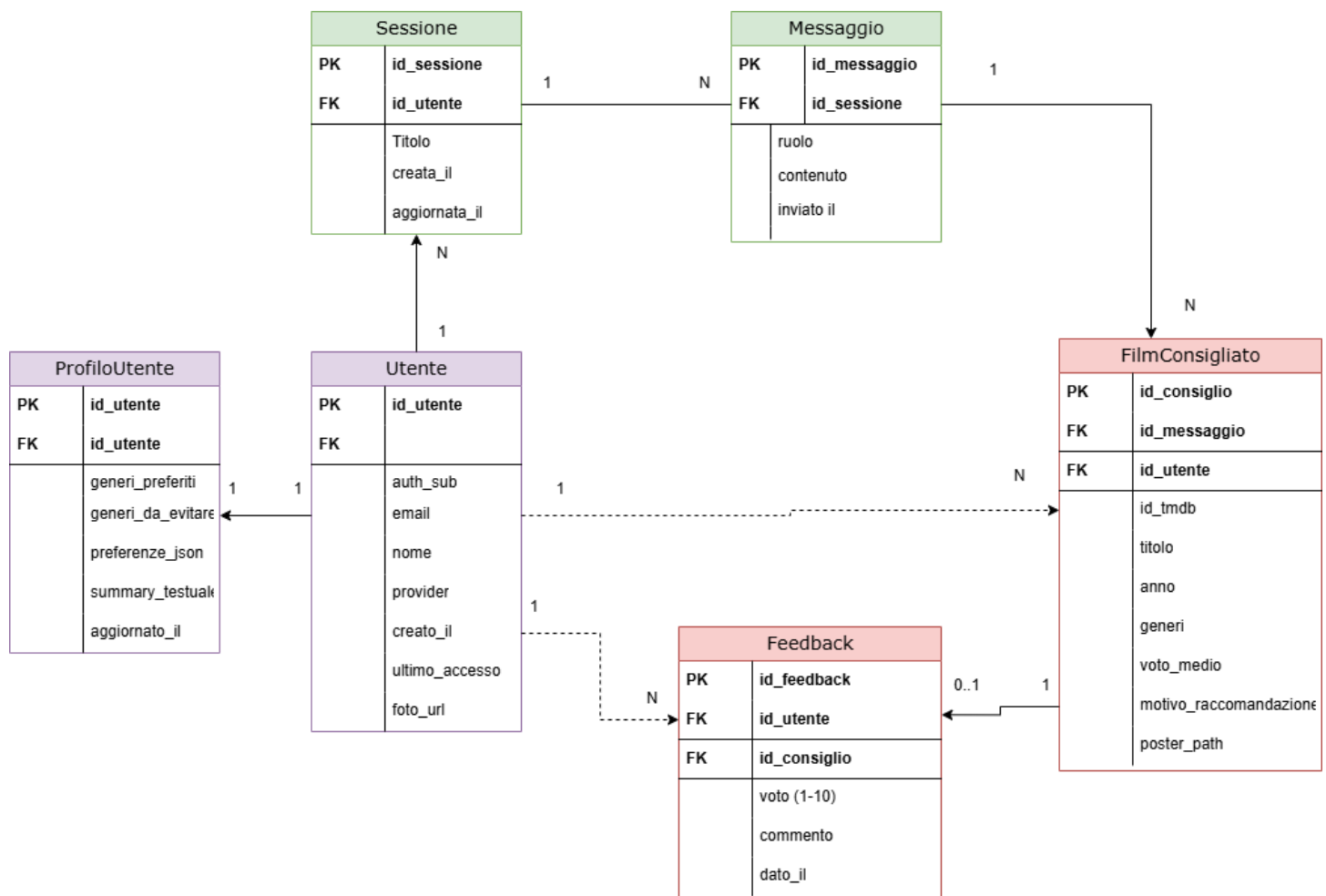
9.1 SQLite + SQLAlchemy ORM

Il database relazionale è SQLite, gestito tramite SQLAlchemy ORM. Le tabelle coprono utenti, sessioni, messaggi, film consigliati, feedback, profilo utente e i film già visti (tabella `FilmVisto`): questi ultimi sono la fonte precisa per non riproporre film già visti o scelti, anche tra sessioni diverse.

Modello dei dati

Tabella	Campi principali	Scopo
Utente	<code>id_utente</code> , <code>auth_sub</code> , <code>email</code> , <code>nome</code> , <code>provider</code>	Anagrafica utenti con supporto Auth0
Sessione	<code>id_sessione</code> , <code>id_utente</code> (FK), <code>titolo</code> , <code>timestamps</code>	Raggruppamento logico dei messaggi
Messaggio	<code>id_messaggio</code> , <code>id_sessione</code> (FK), <code>ruolo</code> , <code>contenuto</code>	Storico completo della conversazione
FilmConsigliato	<code>id_tmdb</code> , <code>titolo</code> , <code>generi</code> (JSON), <code>voto</code> , <code>motivo</code>	Film raccomandati, collegati al messaggio assistente

Feedback	id_utente, id_consiglio (FK), voto (1-10), commento	Valutazione utente dei film consigliati
ProfiloUtente	generi_preferiti (JSON), generi_da_evitare (JSON), preferenze_json, summary_testuale	Profilo stabile delle preferenze
FilmVisto	id_utente, id_tmdb, titolo, fonte, gradito	Film già visti o scelti: fonte precisa per l'esclusione dai consigli, persistente tra le sessioni



9.2 ChromaDB (memoria semantica)

ChromaDB è un database vettoriale persistente. Per ogni utente viene creata una collection dedicata (`memorie_utente_{id_utente}`), che contiene i messaggi di preferenza salvati come embedding testuali.

Ogni documento ChromaDB include metadati strutturati: tipo (genere_preferito, fatto_personale, feedback), fonte, timestamp e id_utente. La funzione `cerca_memorie_utente` recupera le memorie più simili semanticamente al testo corrente, consentendo di arricchire il contesto anche per preferenze espresse in modo indiretto o con parafrasatura.

ChromaDB è complementare a SQLite, non sostitutivo: SQLite rimane la fonte autorevole per i dati precisi (film visti, generi preferiti), mentre ChromaDB potenzia la capacità di recupero semantico per preferenze espresse in linguaggio naturale.

10. Integrazione con Servizi Esterni

10.1 TMDb API

The Movie Database (TMDb) è la fonte di tutti i dati cinematografici. Il modulo `backend/tools/tmdb_tools.py` espone le seguenti funzioni:

- `cerca_film`: ricerca per titolo, usata per trovare il film di riferimento nelle query "simili a".
- `cerca_film_per_genere`: `/discover/movie` con filtri per `id_genere`, `vote_average.gte`, `primary_release_date`.
- `get_film_simili`: `/movie/{id}/recommendations` (non `/similar`), restituisce film raccomandati dall'algoritmo TMDb.
- `get_generi_film`: `/genre/movie/list`, caricato una volta per sessione per mappare nomi a ID.
- `get_movie_details`: `/movie/{id}`, usato per arricchire i dettagli di un film specifico.

Il mapping tra nomi di genere in italiano e ID TMDb avviene tramite `trova_genere_tmdb`, che usa un confronto per sottostringa (ad esempio "musica" viene trovato nell'entry "Music" di TMDb).

10.2 OpenRouter e agno Agent SDK

Il modello LLM utilizzato è `openai/gpt-oss-20b:free` di OpenRouter. Il modulo `llm_config.py` centralizza la creazione del modello agente:

- `set_llm_api_key()`: legge `API_KEY_LLM` (o `OPENROUTER_API_KEY`) dai secrets e la imposta come variabile d'ambiente.
- `crea_model_agente()` / `get_model_id()`: restituiscono l'istanza `OpenRouter(id=model_id)` e l'ID del modello, validi per tutti gli agenti (router, memoria, rerank, chiarimenti).

Il framework open-source agno fornisce un'astrazione unificata per la creazione di agenti LLM con istruzioni strutturate e gestione del contesto. Ogni agente viene istanziato con istruzioni specifiche; il router e il reranker chiedono al modello una risposta JSON, con parsing robusto (`estrai_json_da_testo`) e fallback deterministico se il modello non rispetta il formato.

11. Schemi Pydantic e Contratti tra Agenti

Il modulo `schemas.py` definisce tutti i tipi di dato scambiati tra gli agenti tramite modelli Pydantic. L'utilizzo di Pydantic garantisce validazione automatica, errori leggibili e autocomplete durante lo sviluppo.

Schema	Descrizione
FilmCandidato	Rappresenta un film recuperato da TMDb: <code>tmdb_id</code> , <code>titolo</code> , <code>anno</code> , <code>generi</code> , <code>voto_medio</code> , <code>poster_path</code> , <code>motivo_ricerca</code> , <code>numero_voti</code> .
ConduttoreOutput	Output dell'orchestratore: <code>success</code> , <code>azione</code> , <code>risposta</code> , <code>film_consigliati</code> (lista <code>FilmCandidato</code>), <code>memoria_aggiornata</code> , <code>titolo_chat</code> , <code>richiesta_eseguita</code> (stato di sessione), <code>errors</code> , <code>log_notes</code> .

SearchAgentOutput	Output del SearchAgent: success, film_candidati, pool_candidati (pool completo dei validi), errors, log_notes.
MemoryAgentOutput	Output del MemoryAgent: success, generi_preferiti, generi_da_evitare, summary_testuale e altri campi di memoria.
PreferenzeEstratteMemoria	Risultato intermedio dell'estrazione preferenze: is_memoria, tipo_memoria, generi_preferiti, generi_da_evitare, confidence (0-1).
DecisioneRouter	Output LLM del router: azione (raccomandazione/chiarimento/conversazione_film/domanda_memoria/aggiornamento_memoria/fuori_dominio), tipo_richiesta (NUOVA/CONTINUAZIONE/RAFFINAZIONE), segna_visti_recenti, aggiorna_memoria, film_scelto, risposta_diretta, ricerca (RichiestaRicerca).
RichiestaRicerca	Contratto strutturato conduttore→SearchAgent e stato di sessione: tipo, generi_richiesti/esclusi, voto/anno min-max, film_base, query_tema, tmdb_id_da_evitare, preferenze e feedback del profilo, pagina, candidati_salvati.

12. Supporto IA per lo sviluppo di MovieAgentOS

Durante lo sviluppo di MovieAgentOS abbiamo usato Claude.ai come strumento di supporto per analizzare, testare, correggere e migliorare il codice scritto da noi. Di seguito documentiamo alcuni problemi concreti che abbiamo affrontato con questo aiuto, il modo in cui abbiamo formulato le richieste e il risultato ottenuto nel codice.

Per formulare i prompt abbiamo adottato il framework COSTAR (Context, Objective, Style, Tone, Audience, Response). In tutte le richieste abbiamo mantenuto costanti quattro elementi, descritti qui una volta per non ripeterli: lo Stile richiesto era codice Python pulito, con funzioni semplici e testabili, secondo i principi della buona programmazione e dell'ingegneria del software; il Tono tecnico; l'Audience studenti universitari alle prese con un progetto d'esame; il formato della Risposta codice Python accompagnato da una spiegazione in linguaggio naturale. Il Contesto di base era il sistema multi-agent stesso: un'applicazione Python che dialoga in linguaggio naturale con l'utente e sceglie tramite un LLM che tipo di ricerca di film effettuare. Per ogni problema riportiamo quindi solo l'Obiettivo specifico del prompt e il risultato ottenuto.

12.1 Problema: il sistema proponeva sempre film recenti, indipendentemente dalla richiesta

Obiettivo del prompt: risolvere il fatto che il sistema proponeva sempre film recenti, ignorando i vincoli espressi dall'utente (anno e voto).

Risultato nel codice: abbiamo estratto i vincoli numerici (anno, voto) direttamente dal testo dell'utente con gli estrattori deterministici di linguaggio.py (estrai_anni, estrai_voto_min, estrai_voto_max) e li applichiamo in un solo punto (film_rispetta_vincoli): il modello non può più ignorarli proponendo film generici o recenti.

12.2 Problema: il sistema confondeva le richieste di novità con quelle di film simili

Obiettivo del prompt: evitare che, quando l'utente non indica esplicitamente un titolo di riferimento, il sistema inventi un film da cui partire per una ricerca di "simili".

Risultato nel codice: abbiamo introdotto la funzione `estrai_film_riferimento()` in `linguaggio.py`, che riconosce i riferimenti del tipo "simile a X" e attiva la ricerca per similarità solo in presenza di un film di riferimento esplicito, evitando ricerche "simili" senza un titolo di base.

12.3 Problema: il planner ignorava i numeri indicati nella richiesta e l'LLM inventava i valori

Obiettivo del prompt: impedire che, quando la richiesta contiene vincoli come voto minimo o anno, il modello li ignori o li sbagli — ad esempio restituendo `voto_min` nullo dopo un "voglio film con voto superiore a 7".

Risultato nel codice: abbiamo consolidato l'estrazione dei valori numerici dal testo lato Python invece che delegarla all'LLM, garantendo che l'anno e il voto indicati nella richiesta siano sempre rispettati durante la ricerca.

12.4 Miglioramento della pagina profilo e della lista dei film piaciuti e non piaciuti

Obiettivo del prompt: rendere la pagina profilo chiara e leggibile, mostrando i gusti salvati (generi preferiti e da evitare) e la lista dei film valutati dall'utente, così da fargli capire cosa il sistema ha imparato su di lui e quali film non gli verranno più riproposti. In questo caso il contesto includeva anche l'interfaccia Streamlit e la pagina profilo, oltre alla chat.

Risultato nel codice: abbiamo realizzato le sezioni "I tuoi gusti" e "I tuoi film" della pagina profilo (`profile.py`). La prima mostra in due colonne i generi preferiti e i generi da evitare, con un pulsante per cancellare le preferenze (reset di SQLite e delle memorie ChromaDB). La seconda legge i dati precisi da SQLite tramite `crud.ottieni_gusti_film_utente` e mostra, sempre in due colonne, i film valutati positivamente e negativamente con titolo, anno e generi. Le stesse valutazioni vengono poi usate come criterio nel rerank dei consigli e i film già valutati non vengono mai riproposti.